

# Modelos de Processo de Software

Rebeca Schroeder Freitas

# Introdução

## Objetivos da Aula:

- Introduzir os modelos de ciclo de vida de processo de software
- Relacionar contribuições e problemas dos diversos modelos
- Identificar as melhores práticas e preocupações gerais do processo de software

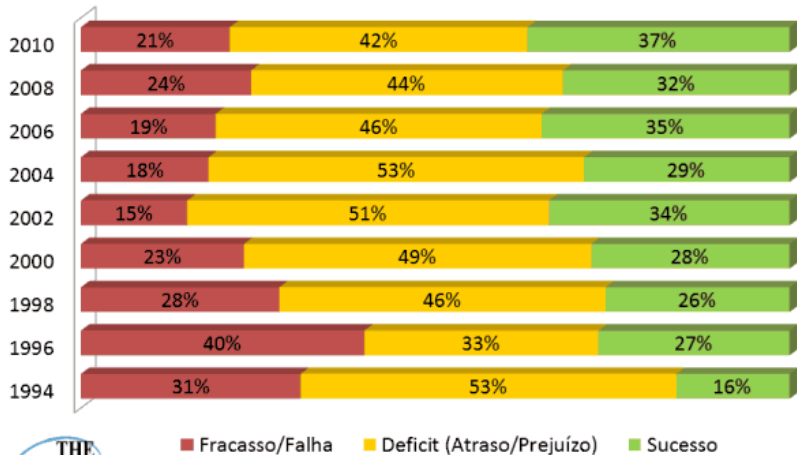
# Plano da Aula

- 1 Introdução
- 2 Modelos Prescritivos
  - Modelos Sequenciais
  - Modelos Incrementais
  - Modelos Evolucionários
- 3 Métodos Ágeis
- 4 Processo Unificado
- 5 Considerações Finais
- 6 Exercícios de Fixação

— Jacobson, Booch e Rumbaugh



# O Projeto de software



Fonte: Standish Group; CHAOS Manifesto 2011, CHAOS Summary for 2010, Extreme CHAOS 2001.

# Conceitos Envolvidos:

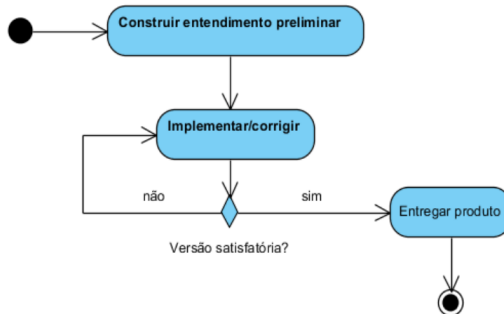
- Papel (Quem):
  - conjunto de responsabilidades para uma pessoa ou grupo
  - exemplo: analista, desenvolvedor, gerente
- Artefato (O quê):
  - subproduto produzido durante o desenvolvimento do software
  - exemplo: documentos, diagramas, código...
- Atividades (Como):
  - unidades de trabalho executadas por um papel
  - produz resultados palpáveis - criação/alteração de artefatos
  - exemplo: gerenciar o escopo do sistema

# Conceitos Envolvidos:

- Disciplinas (Quando):
  - descrevem uma possível abordagem ao problema
  - conjunto de atividades correlacionadas
  - exemplo: Análise, Projeto, Implementação, Teste
- Fase de processo (Resultados parciais):
  - estágio específico de desenvolvimento de um software
  - resultado do produto de uma ou mais disciplinas

# Modelos de Processo de Software

- Representação simplificada e abstrata do processo de software
- Forma geral de comportamento, baseada na qual processos específicos podem ser definidos
- Representa uma tentativa de colocar ordem em um processo inerentemente caótico





# Modelos de Processo de Software

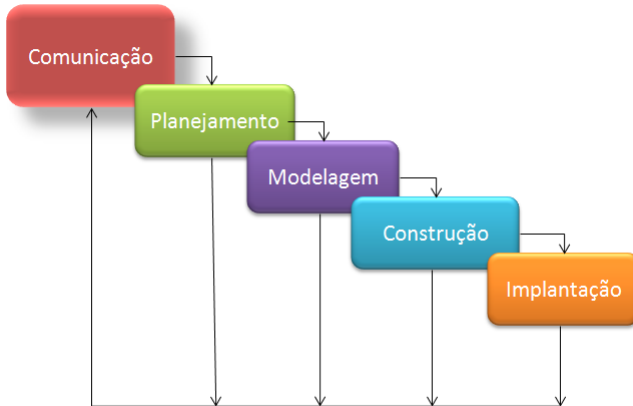
Vantagens em se ter um Modelo de Processo:

- O tempo de treinamento pode ser reduzido
- Produtos podem ser uniformizados
- Resultados:
  - previsíveis
  - gerenciáveis
- Possibilidade de Capitalizar Experiências
  - melhoria contínua do processo
  - melhoria contínua do produto

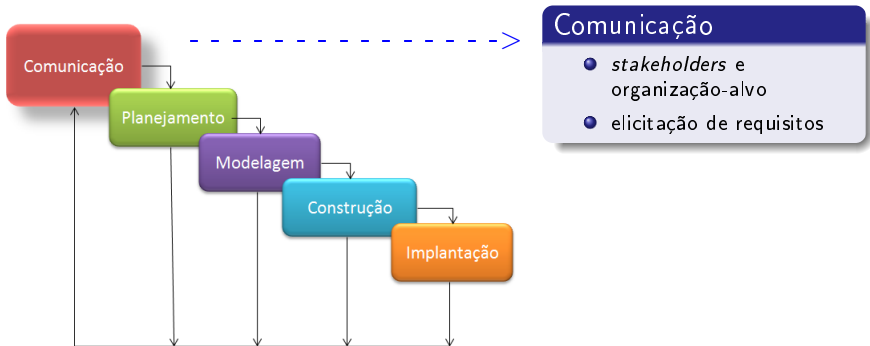
# Plano da Aula

- 1 Introdução
- 2 Modelos Prescritivos
  - Modelos Sequenciais
  - Modelos Incrementais
  - Modelos Evolucionários
- 3 Métodos Ágeis
- 4 Processo Unificado
- 5 Considerações Finais
- 6 Exercícios de Fixação

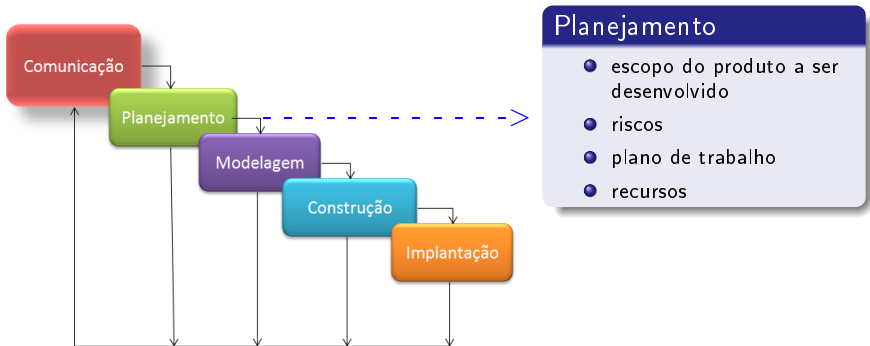
# Modelo Cascata



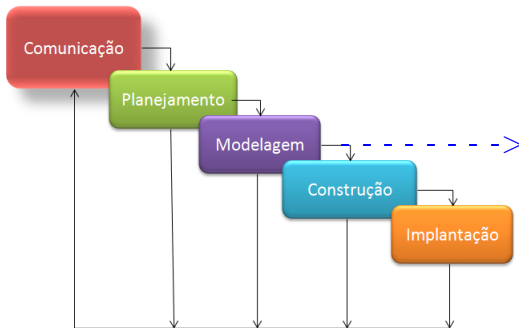
# Modelo Cascata



# Modelo Cascata



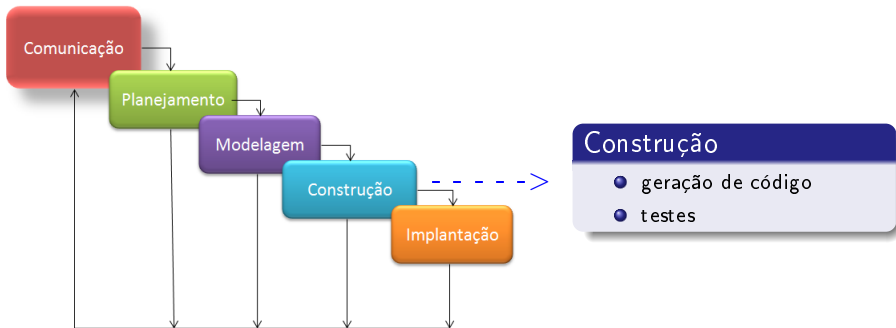
# Modelo Cascata



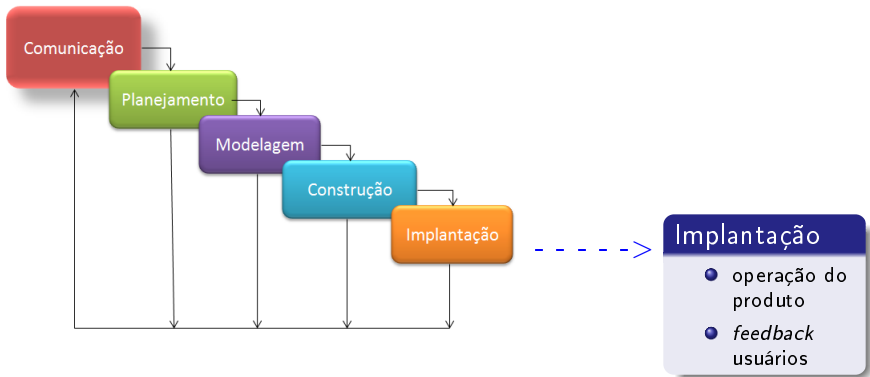
## Modelagem

- análise requisitos
- projeto do sistema
- camada de domínio
- camada de interface

# Modelo Cascata



# Modelo Cascata





# Modelo Cascata

## Contribuições (+)

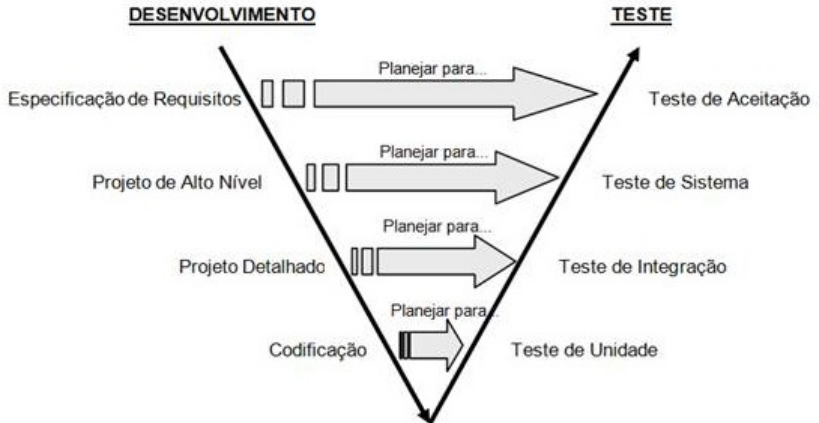
- organização de atividades, disciplina, planejamento e gerenciamento
- marcos
- documentação

## Problemas (-)

- difícil elicitar todos os requisitos logo no início
- projetos reais raramente seguem o fluxo sequencial que o modelo propõe
- uma versão executável do software só fica disponível após a construção

\* Diversas variantes do Modelo Cascata

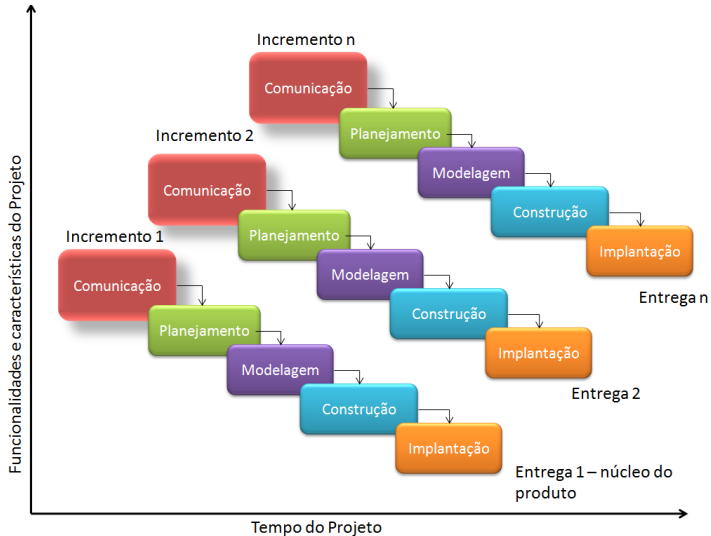
# Modelo Cascata V e W (foco em testes)



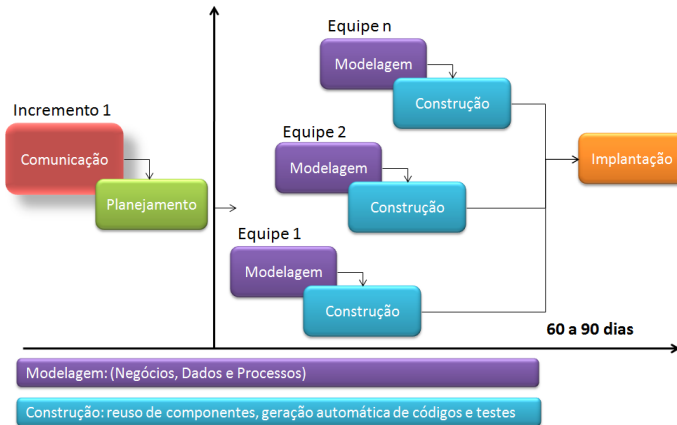
# Plano da Aula

- 1 Introdução
- 2 Modelos Prescritivos
  - Modelos Sequenciais
  - Modelos Incrementais
  - Modelos Evolucionários
- 3 Métodos Ágeis
- 4 Processo Unificado
- 5 Considerações Finais
- 6 Exercícios de Fixação

# Modelo Incremental



# Rapid Application Development (RAD)



# Plano da Aula

- 1 Introdução
- 2 Modelos Prescritivos
  - Modelos Sequenciais
  - Modelos Incrementais
  - Modelos Evolucionários
- 3 Métodos Ágeis
- 4 Processo Unificado
- 5 Considerações Finais
- 6 Exercícios de Fixação

# Modelos Evolucionários

## Contribuições (+)

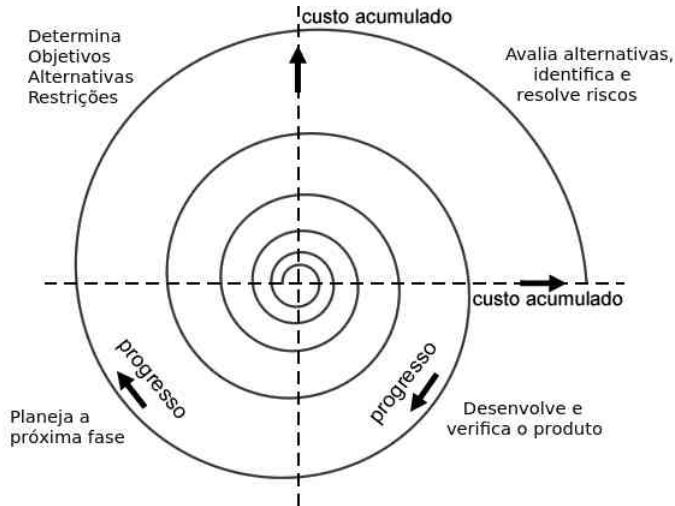
- Tratam a natureza evolutiva do software
  - Requisitos instáveis
  - O conteúdo funcional de um sistema deve crescer continuamente e progressivamente para manter a satisfação do usuário
- Versões progressivamente mais completas:
  - versões de funcionalidades
  - novas funcionalidades completas
- Mitigação de riscos

# Modelos Evolucionários - Prototipação





## Modelos Evolucionários - Espiral



# Modelos Evolucionários

## Problemas (-)

- gerenciamento complexo: número incerto de ciclos
- rápido? / lento?
- foco principal na extensibilidade e não em qualidade
- Prototipação:
  - o cliente vê como uma versão definitiva
  - o desenvolvedor faz concessões impróprias a fim de colocar o protótipo em funcionamento

# Plano da Aula

- 1 Introdução
- 2 Modelos Prescritivos
  - Modelos Sequenciais
  - Modelos Incrementais
  - Modelos Evolucionários
- 3 Métodos Ágeis
- 4 Processo Unificado
- 5 Considerações Finais
- 6 Exercícios de Fixação

# Métodos Ágeis

Manifesto ágil de 2001:

- indivíduos e interações estão acima de processo e ferramentas
- software funcionando está acima de documentação abrangente
- colaboração do cliente está acima de negociação de contrato
- responder as mudanças está acima de seguir um plano

# Métodos Ágeis

Alguns métodos:

- Scrum
- XP
- Crystal (família de métodos)
- FDD - *Feature Driven Development*
- DSDM - *Dynamic Systems Development Method*
- ASD - *Adaptive Software Development*

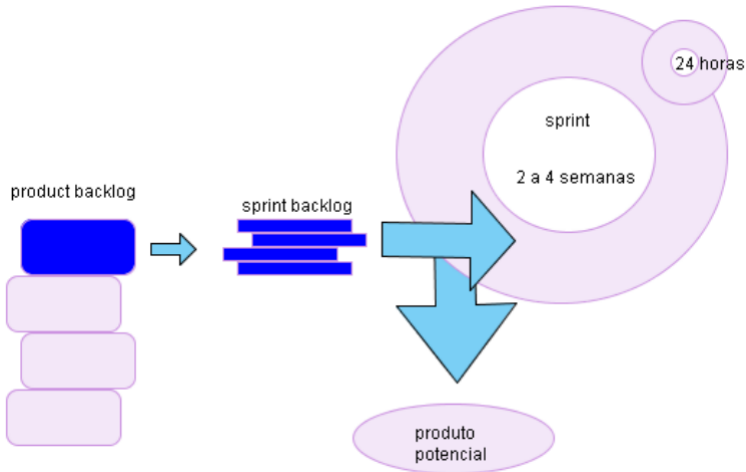
# Métodos Ágeis

Características em comum:

- equipes e processos auto-ajustáveis
- iterativo e incremental
- orientado a cronogramas breves
- pouca documentação
- programação em pares
- código coletivo
- não orientados a ferramentas
- desenvolvimento dirigido por testes

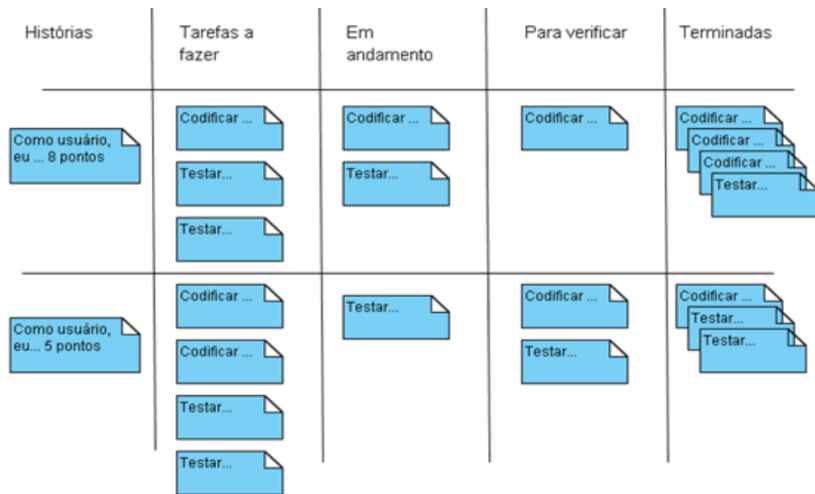
# Scrum

## Ciclo de vida:



# Scrum

## Scrum board:





# Modelos Prescritivos x Métodos Ágeis

| Modelo                     | Foco   |        |        |             |             |            |          |
|----------------------------|--------|--------|--------|-------------|-------------|------------|----------|
|                            | Marcos | Testes | Riscos | Componentes | Incremental | Protótipos | Entregas |
| Cascata                    | ✓      |        |        |             |             |            |          |
| V e W                      | ✓      | ✓      |        |             |             |            |          |
| RAD                        | ✓      |        |        | ✓           | ✓           |            | ✓        |
| Prototipação Evolucionária | ✓      |        | ✓      |             | ✓           | ✓          |          |
| Espiral                    | ✓      | ✓      | ✓      |             | ✓           | ✓          |          |
| Métodos Ágeis              |        | ✓      | ✓      | -           | ✓           | ✓          | ✓        |

- ✓ : implementado
- : parcialmente implementado
- : não implementado

# Modelos Prescritivos x Métodos Ágeis

| Modelo                     | Foco   |        |        |             |             |            |          |
|----------------------------|--------|--------|--------|-------------|-------------|------------|----------|
|                            | Marcos | Testes | Riscos | Componentes | Incremental | Protótipos | Entregas |
| Cascata                    | ✓      |        |        |             |             |            |          |
| V e W                      | ✓      | ✓      |        |             |             |            |          |
| RAD                        | ✓      |        |        | ✓           | ✓           |            | ✓        |
| Prototipação Evolucionária | ✓      |        | ✓      |             | ✓           | ✓          |          |
| Espiral                    | ✓      | ✓      | ✓      |             | ✓           | ✓          |          |
| Métodos Ágeis              |        | ✓      | ✓      | -           | ✓           | ✓          | ✓        |
| <b>Processo Unificado</b>  | ✓      | ✓      | ✓      | ✓           | ✓           | ✓          | ✓        |

## Processo Unificado

- Melhores práticas (Modelos prescritivos)
- Forte interação com usuário

- Dirigido por Casos de Uso
- Baseado em componentes
- Centrado na arquitetura
- Iterativo e incremental

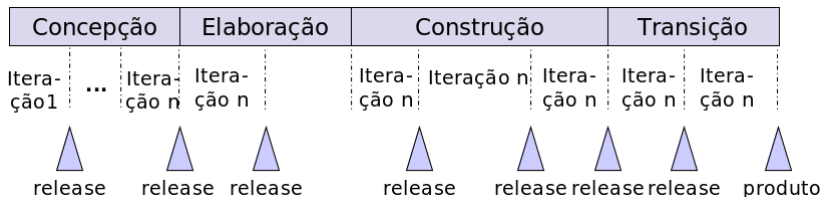
# Plano da Aula

- 1 Introdução
- 2 Modelos Prescritivos
  - Modelos Sequenciais
  - Modelos Incrementais
  - Modelos Evolucionários
- 3 Métodos Ágeis
- 4 Processo Unificado
- 5 Considerações Finais
- 6 Exercícios de Fixação

# Processo Unificado

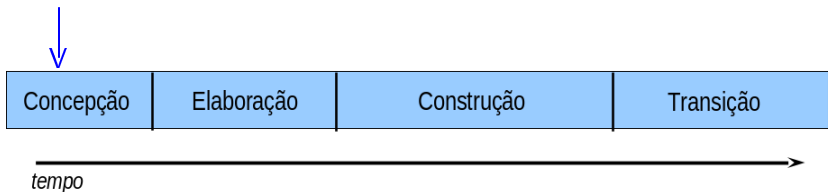
## Fases:

- perspectiva dinâmica
- estágios do desenvolvimento do software
- cada iteração gera um artefato ou um conjunto deles (*release*)



# Processo Unificado

Fases:

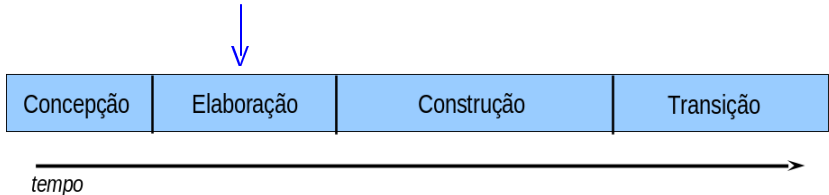


- **Concepção/Iniciação:**

- define o escopo do projeto e sua viabilidade
- deve ser breve: 2 semanas a 2 meses
- Marco: *Lifecycle Objective Milestone (LCO)*

# Processo Unificado

Fases:

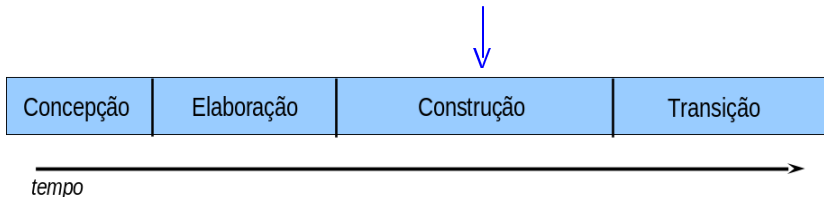


- **Elaboração:**

- detalhar a análise de requisitos (80%)
- pode envolver diversas iterações
- definição estável da arquitetura dos sub-sistemas
- Marco: *Lifecycle Architecture Milestones (LCA)*

# Processo Unificado

## Fases:

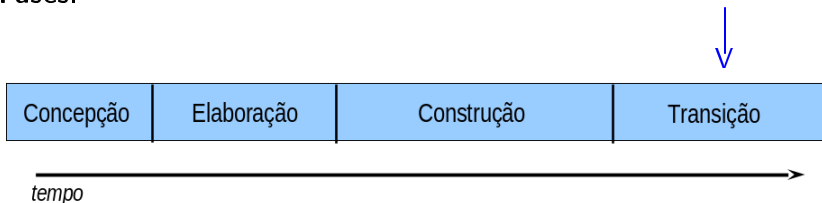


### • Construção:

- geração de código
- descrever e desenvolver requisitos que ainda faltam
- Testes de sistema
- Marco: *Initial Operational Capability (IOC)*

# Processo Unificado

## Fases:



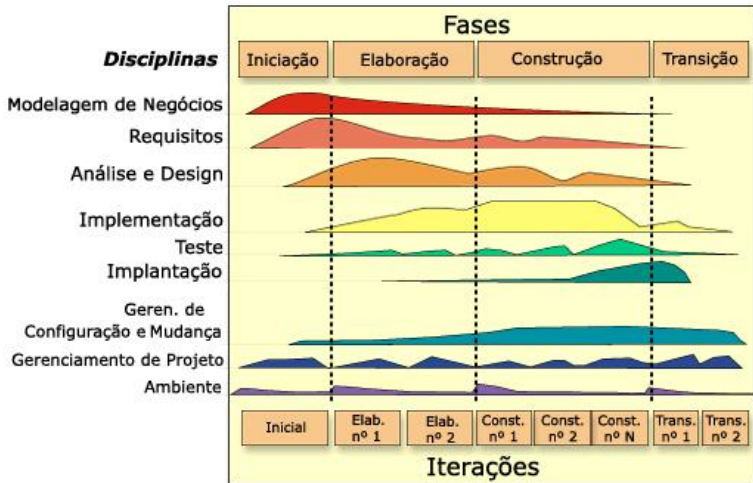
### ● Transição:

- implementação do sistema no ambiente final
- testes de aceitação e operação
- migração de sistemas legados
- treinamento
- adaptações
- Marco: *Product Release Milestone (PR)*



# Processo Unificado

## Disciplinas:



# Rational Unified Process (RUP)

Rational Unified Process (RUP):

- Risco como principal fator de determinação de iterações
- Gerenciar requisitos
- Modelar o software visualmente com diagramas UML
- Controlar as mudanças

\* inclui uma base de dados com *hyperlinks* com vários artefatos e *templates* necessários para usar o modelo.

# Plano da Aula

- 1 Introdução
- 2 Modelos Prescritivos
  - Modelos Sequenciais
  - Modelos Incrementais
  - Modelos Evolucionários
- 3 Métodos Ágeis
- 4 Processo Unificado
- 5 Considerações Finais
- 6 Exercícios de Fixação

# Escolha de um Modelo de Processo de Software

## Processo Prescritivos

- + Marcos bem definidos
- + Documentação
- + Atividades bem definidas
- + Melhor controle
  - Mudanças (em alguns casos)
  - Papel do cliente

## Processos Ágeis

- + Equipes e Processos auto-ajustáveis
- + Cliente é parte da equipe
  - Risco: fator humano
  - Documentação, Processos
  - Simplificação do gerenciamento

## Processo Unificado

- + Capitaliza as melhores práticas
- + Um *framework* de processos
  - Complexidade
  - Investimento em ferramentas
  - Exige experiência da equipe

\* **Métodos Formais:** uma alternativa para sistemas críticos

# Escolha de um Modelo de Processo de Software

- Natureza do projeto e da aplicação:
  - Qual o tamanho do projeto?
  - Quão bem os analistas e o cliente podem conhecer os requisitos do sistema?
  - Quão bem é compreendida a arquitetura do sistema?
  - Qual o grau de risco que este projeto apresenta?
- Métodos e ferramentas a serem usados:
  - Qual o grau de treinamento e adaptação necessários para a equipe?
  - Será desenvolvido um único sistema ou uma família de sistemas semelhantes?
- Controles e produtos que precisam ser entregues:
  - Qual o grau de confiabilidade necessário em relação ao cronograma?
  - Quanto planejamento é efetivamente necessário?

# Referências

-  Pressman, Roger. Software Engineering: A Practitioner's Approach. 7 ed, 2009.
-  SimSE. An Education, Game-based Software Engineering Simulation Environment.  
<http://www.ics.uci.edu/emilyo/SimSE/>. Acesso em 9/4/14.
-  Sommerville, Ian. Engenharia de Software. 8 ed, 2007.
-  The Standish Group. <http://blog.standishgroup.com/>. Acesso em 9/4/14;
-  Wazlawick, Raul Sidnei. Engenharia de Software: Conceitos e Prática. 1 ed, 2013.